PRÁCTICA 6 : *Control de recursos limitados*

OBJETIVO:

1. Implementar la sincronización necesaria entre tareas para el acceso a recursos limitados utilizando la sentencia de **reencolado** en Ada.
2. Utilizar prioridades de tareas en Ada para determinar el orden de ejecución de tareas en colas de espera de entries

PERIODO RECOMENDADO PARA SU REALIZACIÓN: 2 semanas

ENUNCIADO: Aterrizaje de aviones.

En esta práctica vamos a implementar el aterrizaje de los aviones en la pista asignada por la torre de control.

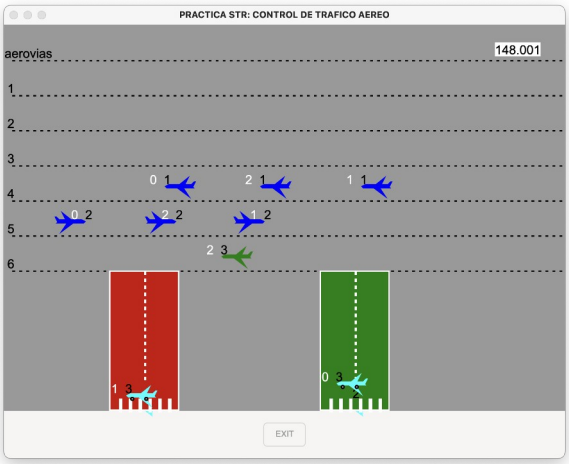


Figura 1. Hay 2 aviones aterrizando en su correspondiente pista. El avión (2-3) en la aerovía 6 tiene concedida la pista 2, pero está esperando al permiso de aterrizaje (no tiene activo el tren de aterrizaje). Los aviones con color azul están esperando permiso de descenso. El resto de aviones que no aparecen ya han aterrizado.

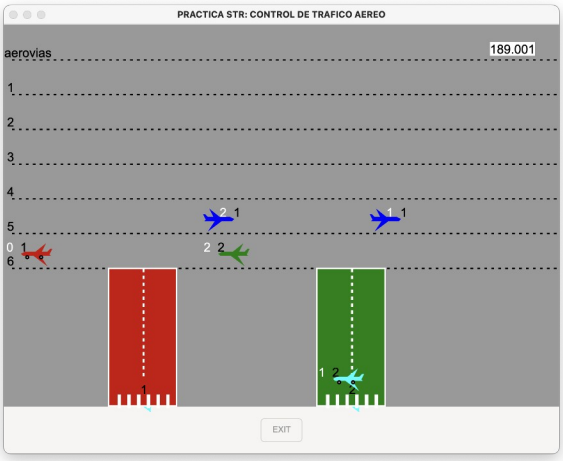


Figura 2. Al avión rojo se le ha concedido permiso de aterrizaje y está en maniobra de aproximación a la pista 1 (ha activado el tren de aterrizaje). El avión verde todavía no tiene concedido el permiso de aterrizaje (su pista está ocupada) y por tanto no tiene activado su tren de aterizaje.

Ayuda para la implementación:
Como tendrás que añadir algún entry a la tarea servidora de la torre de control, es importante el orden en el que escribas los accept dentro de la sentencia select accept … or accept … or accept … or accept … end select;
Ten en cuenta que si las tareas tienen la misma prioridad, como es en nuestro caso, ya que se están creando con la prioridad por defecto, el orden de prioridad para evaluar los accept dentro de la select viene determinado por el orden en el que están escritos (Ver explicación adicional en el Anexo).

PASOS A SEGUIR:

PASO 1: Permiso de aterrizaje

Cuando un avión alcanza la última aerovía, primero debe solicitar a la torre de control una pista de aterrizaje. Mientras no se le asigne una pista, éste continúa volando.

La torre de control irá asignando de forma alternativa las 2 pistas cada vez que le llega una solicitud, sin realizar ninguna comprobación del estado de la pista. Si se asigna la pista 1, el avión cambiará de color a rojo, y si se asigna la pista 2, a color verde, y actualizará el campo 'pista' de su struct.

Después de la asignación de una pista, el avión tiene que solicitar permiso de aterrizaje a la torre de control. Mientras que no se libere la pista asignada al avión, éste continúa volando y la torre de control podrá atender otras peticiones de otros aviones.

Cuando se libere su pista asignada, el avión desplegará el tren de aterrizaje (actualizar a true el campo correspondiente del registro T_RecordAvion) y continuará con la maniobra de aproximación a la pista, descrita en el siguiente paso.

Recomendación para la implementación:
Para simplificar el código del body del tipo tarea avión, es recomendable que definas procedimientos para las distintas acciones que tienen que llevar a cabo los aviones: volar por una aerovía, descender de aerovía, maniobra de aproximación a la pista, aterrizar en una pista y cualquier otra acción que consideres.

PASO 2: Maniobra de aproximación a la pista

El avión debe seguir volando en la última aerovía hasta alcanzar la posición de su pista asignada.

La posición (coordenadas x,y) de cada pista se obtiene usando la función *PKG_graficos*.**Pos_Inicio** con el argumento el número de pista asignada. Hay que comprobar que la zona correspondiente a la coordenada X de la pista es igual a la zona de la coordenada X del avión, para saber si ha alcanzado la pista de aterrizaje. Recuerda que para determinar qué zona del espacio aéreo le corresponde una determinada posición con coordenada 'x', debes usar la función *PKG_graficos*.**Posicion_ZonaEspacioAereo**

Después de alcanzar la pista, se procede a la maniobra de aterrizaje, descrita en el siguiente paso.

PASO 3: Maniobra de aterrizaje en la pista

Antes de iniciar la maniobra de aterrizaje, tienes que hacer desaparecer el avión del espacio aéreo, usando el procedimiento *PKG_graficos*.**Desaparece.** Recuerda que también debes liberar las zonas ocupadas por el avión y decrementar el número de aviones en la aerovía.

Para la maniobra de aterrizaje debes tener en cuenta lo siguiente:

1. Cambia el color del avión a Cyan
2. La velocidad en el eje X (horizontal) debe ser 0 y en el eje Y (vertical) será *pkg_tipos*.**VELOCIDAD_VUELO** (que será la velocidad de entrada a la pista).
3. La posición X de la pista se obtiene con *PKG_graficos*.**Pos_Inicio** pasando como argumento el número de pista. Hay que comprobar que la zona de la posición X de la pista coincide con la zona de la posición X del avión, para saber cuándo debe aparecer en la pista.
4. Usa el procedimiento *pkg_graficos*.**Aparece_En_Pista** para visualizarlo al principio de la pista.
5. Usa la función booleana *pkg_graficos*.**Fin_Aterrizaje** que devolverá true cuando la posición Y del avión que se especifique se corresponda con el final de la pista.
6. Usa el procedimiento *pkg_graficos*.**Reduce_Velocidad_Aterrizaje** para ir desacelerando el avión mientras aterriza (disminuye automáticamente la velocidad en el eje Y)
7. Usa el procedimiento *pkg_graficos*.**Actualiza_Movimiento_En_Pista** para ir moviendo el avión por la pista, después de reducirle la velocidad con el procedimiento anterior.
8. Usa el procedimiento *pkg_graficos*.**Desaparece_En_Pista** de forma similar al que usamos para que desaparezca del espacio aéreo, una vez finalizado el aterrizaje.
9. Usa el mismo retardo de movimiento que tenemos para hacer volar los aviones delay(RETARDO_MOVIMIENTO); cada vez que se reduzca la velocidad y se realice un movimiento.
10. Además, cuando llega al final de la pista, y antes de desaparecer, el avión debe comunicar a la torre de control que abandona la pista, y la torre de control se encargará de liberarla.

PASO 4: Cambios de prioridad de los aviones

En la sentencia select accept … or accept … or accept … or accept … end select;
que utilizamos en la tarea servidora de la torre de control, en el primer accept usa el entry para dar permiso de descenso. Podrás comprobar que el sistema no permite aterrizar aviones **¿por qué?**

Soluciona el problema sin cambiar el orden de los accept. Para ello, debes usar el *pragma Queuing_Policy(Priority_Queuing)* y cambiar dinámicamente la prioridad de los aviones de forma conveniente.

ANEXO

El estándar de Ada especifica que la selección entre varias alternativas abiertas es "arbitraria". Sin embargo, arbitrario no significa aleatorio. Para un compilador, generar una selección verdaderamente aleatoria en tiempo de ejecución consumiría tiempo y recursos innecesarios. Por pura eficiencia, compiladores como GNAT resuelven esta ambigüedad leyendo el árbol de sintaxis de arriba a abajo y ejecutando el primer accept que encuentran con tareas pendientes.

El problema de basar la lógica de tu proyecto en este comportamiento es que tu código deja de ser robusto y portable. Si en el futuro compilas ese mismo programa con otro compilador comercial (como ObjectAda o ApexAda), cambias de arquitectura o incluso actualizas la versión de GNAT, el orden de atención podría cambiar silenciosamente. Esto rompería el funcionamiento de tu sistema sin generar ningún error o advertencia de compilación.

Si tu diseño exige estrictamente que una cola se atienda antes que otra, debes forzar ese orden mediante el código, no mediante la posición del texto.

Una de lograr esto es utilizar el atributo 'Count en las guardas lógicas de tu bloque select. Por ejemplo, puedes impedir que se atienda la cola de menor prioridad mientras que existan llamadas pendientes en la principal:

```
select
   when Entrada_Prioritaria'Count = 0 =>
      accept Entrada_Secundaria;
or
      accept Entrada_Prioritaria;
end select;
```

Así garantizas que el programa se comporte exactamente igual en cualquier entorno.

Otra forma es usando asignación de prioridades a las tareas y cambiando la política de ordenación de las llamadas pendientes en los entries.